

Terraform

▼ Terraform

Instalación y configuración básica

Estructura básica de un proyecto

¿Qué es un proyecto en Terraform?

Ejemplo de `main.tf` básico

Archivo `variables.tf` : definición de entradas

Archivo `terraform.tfvars` : valores asignados

Archivo `outputs.tf` : resultados exportables

Separación de lógica por archivos

Organización recomendada por componente

Estructura de carpetas para múltiples entornos

▼ Lenguaje HCL

¿Qué es HCL?

Estructura general de un bloque HCL

Tipos de datos en HCL

Comentarios en HCL

Interpolaciones y expresiones

Operadores en HCL

▼ Terraform CLI

Comando `terraform init`

Comando `terraform plan`

Comando `terraform apply`

Comando `terraform destroy`

Comando `terraform validate`

Comando `terraform fmt`

Comando `terraform show`

Comando `terraform output`

Comando `terraform graph`

Comando `terraform console`

▼ Tipos de variables en Terraform

Input variables: definición

Input variables: asignación

Uso de variables dentro del código

Ejemplo completo con variables – AWS

Local values (`locals`)

Uso combinado de `locals` y `variables`

Outputs en Terraform

▼ Providers, Resources, DataSources

Providers

▼ Resources

Estructura de un bloque `resource`

Uso de referencias entre recursos

▼ Data sources

Ejemplo de data source – AWS

Buenas prácticas con providers, resources y data sources

Ejemplo combinado de resource y data source

▼ Mapeo y gráfico de dependencias y grafico

Uso de `depends_on` para dependencias explícitas

▼ Visualización del grafo de recursos

Interpretación del grafo generado

▼ Estados (State)

Estructura del archivo `terraform.tfstate`

Ubicación del archivo de estado

▼ Gestión del estado

Ejemplo: backend remoto en AWS (S3 + DynamoDB)

Inicialización del backend remoto

Bloqueo del estado

Comandos útiles para gestionar el estado

Sensibilidad y seguridad del estado

▼ Workspaces

Comandos básicos de Workspaces

Ejemplo de uso de workspaces para entornos

Integración de workspace en la configuración

Consideraciones al usar workspaces

▼ Módulos, Expresiones y Funciones

▼ Módulos reutilizables

Estructura interna de un módulo

Ejemplo completo de módulo en AWS

Módulos anidados y reutilización

▼ Expresiones

Tipos de expresiones

Expresiones condicionales

Expresiones `for` y `for_each`

Uso avanzado de `for` en objetos

▼ Funciones

Funciones con colecciones: ejemplo práctico

Uso de funciones con condiciones

Composición de funciones y expresiones

Buenas prácticas con expresiones y funciones

▼ Provisioners

Tipos de provisioners

Ejemplo de `local-exec`

Ejemplo de `remote-exec`

Provisioner `file`

Provisioners y dependencias implícitas

Buenas prácticas con provisioners



Nota

Recuerda que con `aws sts get-caller-identity` puedes verificar que estás utilizando el **rol de laboratorio** AWS.

Instalación y configuración básica

El repositorio para Amazon Linux 2023 sería...

<https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo>

Por tanto, los comandos para instalar Terraform en Amazon Linux 2023 serían:

```
sudo dnf install -y yum-utils
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo dnf install -y terraform
```

En lugar de `sudo dnf install -y terraform` también podemos usar `sudo yum -y install terraform`, pero `dnf` es el gestor de paquetes recomendado para Amazon Linux 2023.

Estructura básica de un proyecto

¿Qué es un proyecto en Terraform?

Un proyecto en Terraform representa un conjunto de configuraciones organizadas para aprovisionar una infraestructura. El núcleo de un proyecto consiste en una o varias configuraciones `.tf` que definen recursos, variables, outputs, y proveedores.

Todo lo que se encuentra en una misma carpeta es interpretado por Terraform como un único módulo raíz. Estos archivos `.tf` se procesan en orden lógico, no alfabético ni por nombre de archivo, lo cual permite dividir la configuración en múltiples archivos sin afectar su funcionamiento.

■ Estructura general de configuración

Un proyecto típico suele tener esta estructura mínima:

```

📁 Proyecto
├─ main.tf – Recursos principales
├─ variables.tf – Declaración de variables
├─ outputs.tf – Outputs que expone el proyecto
└─ terraform.tfvars – Valores concretos para las variables
```

Cada archivo tiene un propósito específico, pero todos se combinan como una sola unidad de ejecución.

■ Guía de estilo de archivos

Ejemplo de `main.tf` básico

Un archivo `main.tf` define recursos y el proveedor. Ejemplo para AWS:

```
provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "example" {
  bucket = "mi-bucket-ejemplo"
  acl    = "private"
}
```

Este archivo puede contener uno o muchos recursos, o incluso incluir los bloques de variables directamente.

■ Ejemplo con AWS

Archivo `variables.tf` : definición de entradas

Las variables se declaran usando bloques `variable` . Ejemplo:

```
variable "region" {  
    description = "Región de despliegue"  
    type        = string  
    default     = "us-east-1"  
}  
  
variable "project_name" {  
    description = "Nombre del proyecto"  
    type        = string  
}
```

Estas variables pueden ser utilizadas dentro de `main.tf` con `var.nombre_variable` .

■ Variables en Terraform

Archivo `terraform.tfvars` : valores asignados

Puedes usar `terraform.tfvars` para definir los valores concretos que usarán las variables:

```
region        = "us-west-1"  
project_name  = "demo"
```

Terraform detecta este archivo automáticamente, y lo aplica al ejecutar `plan` o `apply` .

También puedes usar `*.auto.tfvars` , que siguen el mismo propósito pero permiten múltiples archivos.

■ Asignación de variables

Archivo `outputs.tf` : resultados exportables

Los outputs exponen información útil al final del `apply` , o para pasar valores entre módulos.

```
output "resource_group_name" {  
  value      = azure_rm_resource_group.rg.name  
  description = "Nombre del resource group creado"  
}
```

Pueden marcarse como `sensitive` si contienen datos sensibles que no deben mostrarse.

Outputs

Separación de lógica por archivos

Aunque se suelen usar archivos como `main.tf` , `variables.tf` , etc., en realidad no es obligatorio. Terraform procesa todos los archivos `.tf` juntos.

Puedes, por ejemplo, separar tu infraestructura así:

```
📁 Proyecto  
├─ provider.tf – Configuración del proveedor  
├─ networking.tf – Recursos de red (VPC, subredes...)  
├─ compute.tf – Recursos de cómputo (instancias...)  
└─ storage.tf – Recursos de almacenamiento
```

Esta organización ayuda a mantener limpio el proyecto conforme crece.

Convenciones de estructura

Organización recomendada por componente

Una recomendación habitual es agrupar por tipo de recurso:

- `networking.tf` : VPC, subnets, gateways
- `compute.tf` : EC2, Azure VMs
- `database.tf` : RDS, Azure SQL
- `security.tf` : IAM, NSG

Esto mejora la claridad y el mantenimiento del proyecto, especialmente en equipos grandes.

■ Estructura modular y escalable

Estructura de carpetas para múltiples entornos

Otra estructura habitual es separar por entorno:

```

📁 Proyecto
├─ dev/
│   ├── main.tf – Recursos del entorno de desarrollo
│   └─ terraform.tfvars – Variables para desarrollo
├─ prod/
│   ├── main.tf – Recursos del entorno de producción
│   └─ terraform.tfvars – Variables para producción
├─ modules/
│   └─ vpc/ – Módulo reutilizable de red (VPC)
```

Cada carpeta contiene una configuración idéntica pero con valores distintos, permitiendo despliegues paralelos por entorno.

■ Buenas prácticas de estructura por entorno

Lenguaje HCL

¿Qué es HCL?

HCL (HashiCorp Configuration Language) es el lenguaje de configuración utilizado por Terraform. Es un lenguaje **declarativo**, estructurado por bloques y diseñado para ser legible por humanos. Aunque tiene una sintaxis específica, también admite interpolaciones y expresiones lógicas.

El código en HCL suele estar organizado por **bloques**, con llaves y pares `clave = valor`. Un ejemplo típico sería un recurso cloud:

```
resource "aws_s3_bucket" "ejemplo" {
  bucket = "mi-bucket"
  acl    = "private"
}
```

Estructura general de un bloque HCL

Los bloques de HCL tienen esta forma:

```
<tipo> "<nombre_proveedor>" "<nombre_local>" {  
  argumento1 = valor  
  argumento2 = valor  
}
```

Por ejemplo:

```
resource "azurerm_resource_group" "main" {  
  name      = "rg-ejemplo"  
  location = "westeurope"  
}
```

Los bloques pueden contener **atributos** (pares clave-valor), **bloques anidados** (como `tags` , `ingress` , etc.) y **metaargumentos** como `depends_on` , `count` , `for_each` .

Bloques y estructura HCL

Tipos de datos en HCL

HCL soporta varios tipos de datos básicos:

- **string**: "texto"
- **number**: 42 , 3.14
- **bool**: true / false
- **list**: ["a", "b", "c"]
- **map**: { key1 = "value1", key2 = "value2" }
- **tuple**: [true, 42, "hello"]
- **object**: { name = "Juan", edad = 30 }

Se pueden declarar tipos explícitamente en variables:

```
variable "regiones" {  
  type = list(string)  
}
```


Comentarios en HCL

Puedes documentar tu código usando comentarios:

```
# Comentario de una línea
```

```
/*  
Comentario de  
varias líneas  
*/
```

Es recomendable comentar bloques complejos o explicar decisiones de infraestructura para otros miembros del equipo o para mantenimiento futuro.

Interpolaciones y expresiones

Puedes referenciar otros valores usando la sintaxis `${...}` :

```
resource "aws_instance" "web" {  
  tags = {  
    Name = "${var.entorno}-web"  
  }  
}
```

Desde Terraform 0.12, ya no es necesario `${}` en muchos casos. Puedes usar directamente:

```
Name = var.entorno
```

Operadores en HCL

HCL incluye operadores lógicos y de comparación:

- Comparación: `==` , `!=` , `>` , `<` , `>=` , `<=`
- Booleanos: `&&` (and), `||` (or), `!` (not)

- Concatenación: "prefix-`${var.nombre}`"

Ejemplo:

```
locals {  
  es_produccion = var.env == "prod"  
}
```

■ Expresiones condicionales

Terraform CLI

Terraform CLI (Command Line Interface) es la principal herramienta para interactuar con proyectos Terraform. Permite desde la inicialización del entorno, validación y aplicación de cambios hasta el manejo de estado y workspaces.

Los comandos básicos se ejecutan desde la raíz del proyecto donde se ubican los archivos `.tf`. Algunos de los más usados son:

- `terraform init`
- `terraform plan`
- `terraform apply`
- `terraform destroy`
- `terraform validate`
- `terraform fmt`

■ Comandos de Terraform CLI

Comando `terraform init`

```
terraform init
```

Este comando inicializa el directorio de trabajo de Terraform. Realiza:

- Descarga de proveedores especificados (ej. AWS, AzureRM).
- Creación del directorio `.terraform/`.
- Validación de configuración del backend si se usa uno remoto.

Debe ejecutarse siempre al comenzar un proyecto o tras modificar el `provider` o el `backend`.

■ `init`

Comando terraform plan

```
terraform plan
```

Este comando **simula** los cambios que se aplicarían sin realizarlos. Es muy útil para:

- Ver qué recursos se crearán, destruirán o modificarán.
- Revisar diferencias entre el estado actual y la configuración `.tf`.

Puedes pasar variables:

```
terraform plan -var="region=us-east-1"
```

O usar un archivo `.tfvars`:

```
terraform plan -var-file="dev.tfvars"
```

 plan

Comando terraform apply

```
terraform apply
```

Aplica los cambios necesarios para alcanzar el estado deseado definido en los archivos `.tf`. Se recomienda ejecutar `terraform plan` primero para validar los cambios.

Puedes automatizar la confirmación:

```
terraform apply -auto-approve
```

También puedes aplicar un plan guardado:

```
terraform apply tfplan
```

 apply

Comando terraform destroy

```
terraform destroy
```

Este comando destruye todos los recursos definidos en el proyecto. Es útil para entornos temporales como `dev` o `test`.

Puedes evitar la confirmación interactiva con:

```
terraform destroy -auto-approve
```

⚠ Usa con precaución: borra todos los recursos gestionados por Terraform.

```
 destroy
```

Comando terraform validate

```
terraform validate
```

Valida la sintaxis de los archivos `.tf`. No accede a los proveedores ni modifica nada. Útil para detectar errores básicos de estructura y lógica.

Se recomienda usarlo antes de `plan` o como paso en pipelines CI/CD.

```
 validate
```

Comando terraform fmt

```
terraform fmt
```

Formatea automáticamente el código Terraform siguiendo la convención oficial. Asegura consistencia y mejora la legibilidad del proyecto.

Puedes aplicarlo de forma recursiva:

```
terraform fmt -recursive
```

```
 fmt
```

Comando terraform show

```
terraform show
```

Muestra el contenido del archivo de estado `.tfstate`. Útil para visualizar qué recursos han sido creados, sus atributos y relaciones actuales.

Puedes exportarlo en formato legible o JSON:

```
terraform show -json > estado.json
```

 show

Comando terraform output

```
terraform output
```

Muestra los outputs definidos tras una ejecución. Puedes acceder a un output específico con:

```
terraform output nombre_output
```

También puedes exportarlos en JSON:

```
terraform output -json
```

 output

Comando terraform graph

```
terraform graph
```

Genera un grafo de dependencias entre recursos. Se puede renderizar con Graphviz:

```
terraform graph | dot -Tpng > dependencias.png
```

Ideal para visualizar la estructura de tu infraestructura.

Comando terraform console

terraform console

Abre una consola interactiva para evaluar expresiones Terraform. Útil para pruebas rápidas con variables, funciones o outputs.

Ejemplo:

```
> upper("dev")
"DEV"
```

Tipos de variables en Terraform

Terraform permite definir **valores reutilizables** mediante distintos mecanismos:

- **Input variables:** se definen con `variable` y permiten pasar datos externos al módulo.
- **Local values:** permiten definir valores derivados o intermedios con `locals`.
- **Outputs:** exponen información útil hacia fuera del módulo.

Estos mecanismos son fundamentales para reutilizar código, separar la lógica y conectar módulos entre sí.

Input variables: definición

Se definen en el archivo `variables.tf` y se referencian como `var.nombre`.

```
variable "region" {
  type      = string
  description = "Región donde desplegar"
  default    = "us-east-1"
}

variable "tags" {
  type = map(string)
}
```

Pueden tener `default`, un `description` y un `type`. También pueden ser requeridas si no hay valor por defecto.

Definición de variables

Input variables: asignación

Puedes asignar valores de tres maneras:

1. Archivo `.tfvars`:

```
region = "eu-west-1"
```

2. Línea de comandos:

```
terraform apply -var="region=eu-west-1"
```

3. Variables de entorno:

```
export TF_VAR_region="eu-west-1"
```

Terraform aplica esta prioridad: CLI > archivo `.tfvars` > valores por defecto.

Asignar variables

Uso de variables dentro del código

Una vez declaradas, puedes usarlas así:

```
resource "aws_instance" "web" {  
  ami          = var.ami_id  
  instance_type = var.tipo  
}
```

También puedes construir cadenas con ellas:

```
tags = {  
  Name = "${var.entorno}-web"  
}
```

O bien directamente (Terraform >= 0.12):

```
Name = var.entorno
```

Uso de variables

Ejemplo completo con variables – AWS

```
variable "bucket_name" {  
  type        = string  
  description = "Nombre del bucket"  
}  
  
resource "aws_s3_bucket" "ejemplo" {  
  bucket = var.bucket_name  
  acl    = "private"  
}
```

Puedes pasar `bucket_name` desde CLI, `.tfvars` o variable de entorno. Este enfoque evita hardcoding y permite reutilización.

Variables con AWS

Local values (locals)

Sirven para definir valores derivados o intermedios que no dependen del entorno externo.


```
locals {  
  prefijo = "demo"  
  nombre_bucket = "${local.prefijo}-bucket"  
}
```

Se accede con `local.nombre` . Son útiles para simplificar expresiones repetitivas o construir nombres dinámicos.

■ Valores locales

Uso combinado de locals y variables

```
variable "entorno" {  
  type = string  
}  
  
locals {  
  prefijo = "${var.entorno}-infra"  
}  
  
resource "aws_s3_bucket" "logs" {  
  bucket = "${local.prefijo}-logs"  
}
```

Este patrón se usa para construir recursos a partir de inputs + lógica local.

■ Composición con locals

Outputs en Terraform

Outputs permiten exponer información tras el `apply` :

```
output "bucket_name" {  
  value      = aws_s3_bucket.logs.bucket  
  description = "Nombre del bucket creado"  
}
```

Se muestran al final de `terraform apply` , y también se pueden exportar en JSON.

Los outputs pueden marcarse como `sensitive` para ocultarlos.

Providers, Resources, DataSources

Providers

Un **provider** es un plugin que permite a Terraform interactuar con APIs de servicios como AWS, Azure, GCP, Kubernetes, etc. Define el conjunto de recursos y data sources disponibles para ese servicio.

Para usar un provider, se debe declarar explícitamente:

```
provider "aws" {  
    region = "us-east-1"  
}
```

Terraform descarga el provider necesario durante `terraform init`.

Resources

Un **recurso** en Terraform representa un objeto gestionado en la infraestructura, como una instancia EC2, un grupo de recursos, una VNet o un bucket S3.

Los recursos se declaran con:

```
resource "<proveedor>_<tipo>" "<nombre_local>" {  
    # Configuración  
}
```

Ejemplo básico con AWS:

```
resource "aws_s3_bucket" "logs" {  
    bucket = "mi-bucket-logs"  
    acl    = "private"  
}
```

Estructura de un bloque resource

Un bloque resource contiene:

- **Argumentos obligatorios:** definidos por el proveedor (ej. bucket , name , location).
- **Atributos opcionales:** como etiquetas (tags) o parámetros adicionales.
- **Bloques anidados:** para configuraciones específicas (ej. versioning , ingress , timeouts).
- **Meta-argumentos:** como depends_on , count , for_each .

Sintaxis de recursos

Uso de referencias entre recursos

Puedes usar los atributos de un recurso como entrada de otro:

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "subnet" {  
  vpc_id      = aws_vpc.main.id  
  cidr_block = "10.0.1.0/24"  
}
```

Terraform infiere las dependencias automáticamente gracias a estas referencias.

Referencias entre recursos

Data sources

Un **data source** permite obtener información existente fuera del control directo de Terraform, como:

- Una imagen de máquina existente (AMI en AWS, imagen de VM en Azure).
- Un grupo de recursos ya creado.
- Una red virtual externa.

Esto se hace con bloques data .

Data sources

Ejemplo de data source – AWS

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-20.04-amd64-*"]
  }

  owners = ["099720109477"]
}
```

Este bloque obtiene la última AMI de Ubuntu publicada por Canonical. Luego puedes usar `data.aws_ami.ubuntu.id` en otros recursos.

 [AMI data source](#)

Buenas prácticas con providers, resources y data sources

- Acuérdate de declarar los `provider` en un solo lugar.
- Utiliza `data` cuando necesites leer recursos externos sin crearlos.
- Es siempre mejor usar referencias (`resource.x.y`) en lugar de duplicar valores.
- Documenta tus `resource` con comentarios detallados de su utilidad.
- No abuses de `count` y `for_each` sin necesidad: pueden generar estructuras difíciles de leer.

 [Documentación general](#)

Ejemplo combinado de resource y data source

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-20.04-amd64-*"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  tags = {
    Name = "web-instance"
  }
}
```

El `data` obtiene la AMI más reciente y el `resource` la utiliza para crear una instancia EC2. Este patrón es muy común para desplegar máquinas con imágenes estándar.

Uso conjunto de data y resource

Mapecto y gráfico de dependencias y grafico

Terraform construye un grafo interno de dependencias entre recursos. Esto le permite calcular el orden correcto de creación, modificación o destrucción de recursos, **sin necesidad de instrucciones explícitas**.

Las dependencias se infieren automáticamente cuando un recurso hace referencia a otro, por ejemplo:

```
resource "aws_subnet" "sub" {
  vpc_id = aws_vpc.main.id
}
```

En este ejemplo, Terraform sabe que `aws_vpc.main` debe crearse antes que `aws_subnet.sub`.

Uso de `depends_on` para dependencias explícitas

En algunos casos, Terraform no puede detectar una dependencia automática. Para esos casos se puede usar `depends_on` :

```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
  depends_on    = [aws_security_group.web_sg]
}
```

Esto obliga a Terraform a esperar hasta que se cree `web_sg` antes de lanzar la instancia, aunque no haya una referencia directa.

■ `depends_on`

Visualización del grafo de recursos

Terraform puede generar un grafo visual de las relaciones entre recursos. Para ello se usa:

```
terraform graph | dot -Tpng > grafo.png
```

Esto requiere tener instalado **Graphviz** (`dot`). El archivo `grafo.png` mostrará un diagrama con las dependencias que Terraform ha calculado.

■ `terraform graph`

Interpretación del grafo generado

El grafo contiene nodos y aristas:

- Cada nodo representa un recurso, módulo o proveedor.
- Las flechas indican dependencia ($A \rightarrow B$ significa que B depende de A).
- Los recursos creados por módulos aparecen con un prefijo como `module.<nombre>` .

Este grafo es útil para **entender la lógica de ejecución** de Terraform, especialmente en proyectos grandes.

■ Ejemplo de grafo

Estados (State)

Terraform mantiene un archivo de estado (`terraform.tfstate`) que **representa el estado actual de la infraestructura** gestionada. Este archivo es esencial para:

- Rastrear los recursos creados y sus propiedades
- Comparar configuraciones durante `plan`
- Conectar recursos entre módulos

Sin el estado, Terraform no puede determinar qué cambiar.

■ Gestión del estado

Estructura del archivo `terraform.tfstate`

El archivo `terraform.tfstate` está en formato JSON y contiene:

- Lista de recursos creados
- Atributos actuales de cada recurso
- Identificadores reales (IDs en AWS o Azure)
- Dependencias entre recursos

Este archivo se actualiza con cada `apply` y debe mantenerse sincronizado con la infraestructura real.

■ Formato de estado

Ubicación del archivo de estado

Por defecto, `terraform.tfstate` se guarda localmente en el mismo directorio del proyecto. Sin embargo, esto **no es recomendable para entornos compartidos**.

Ejemplo:

```
.
├── main.tf
└── terraform.tfstate
```

Para equipos o pipelines CI/CD, se recomienda **almacenamiento remoto**.

■ Estado local vs remoto

Gestión del estado

Terraform permite almacenar el estado en backends remotos como:

- AWS S3 (+ DynamoDB para bloqueo)
- Azure Blob Storage
- GCS (Google Cloud)
- Consul
- Terraform Cloud

Esto mejora la **colaboración, versionado y seguridad** del estado.

■ Backends remotos

Ejemplo: backend remoto en AWS (S3 + DynamoDB)

```
terraform {  
  backend "s3" {  
    bucket      = "mi-bucket-terraform"  
    key         = "infraestructuras/estado.tfstate"  
    region      = "us-east-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

Este backend almacena el estado en S3 y usa DynamoDB para evitar conflictos simultáneos.

■ Backend S3

Inicialización del backend remoto

Tras configurar un backend, se debe ejecutar:

```
terraform init
```

Terraform detecta la configuración del backend y:

- Solicita migrar el estado local (si existía)
- Crea el contenedor remoto si no existe
- Sincroniza el archivo `.tfstate` remoto

Warning

Este paso es necesario tras cualquier cambio en la configuración del backend.

 `terraform init` [con backends](#)

Bloqueo del estado

Cuando se usa un backend como S3 + DynamoDB o Terraform Cloud, se habilita **lockeo automático del estado**.

Esto previene que múltiples usuarios modifiquen el estado al mismo tiempo, evitando corrupciones.

 En backends locales no hay bloqueo.

 [Bloqueo de estado](#)

Comandos útiles para gestionar el estado

- `terraform state list` : muestra todos los recursos conocidos
- `terraform state show <recurso>` : detalles de un recurso
- `terraform state mv` : mueve recursos entre módulos
- `terraform state rm` : elimina un recurso del estado (sin destruirlo)

Estos comandos permiten mantener y depurar manualmente el archivo de estado.

 [Gestión manual del estado](#)

Sensibilidad y seguridad del estado

El archivo `terraform.tfstate` puede contener **valores sensibles** como contraseñas, tokens o secretos. Por eso se recomienda:

- Usar almacenamiento remoto cifrado (S3 con KMS, Blob Storage con TLS)
- No subir el `.tfstate` al control de versiones
- Marcar outputs como `sensitive = true` para ocultarlos
- Utilizar servicios concretos del provider (como Secrets Manager en AWS), aunque eso reduce la portabilidad

 [Protección del estado](#)

Workspaces

Un **workspace** es un entorno lógico de ejecución que permite mantener **varios estados separados** usando la misma configuración de Terraform.

Esto permite, por ejemplo, usar la misma infraestructura base (`.tf`) pero con estados distintos para `dev` , `staging` o `prod` , sin necesidad de duplicar archivos.

■ Workspaces

Cada workspace tiene su propio archivo `terraform.tfstate` . La configuración `.tf` es la misma, pero los valores y recursos aplicados se gestionan de forma independiente.

Terraform crea por defecto un workspace llamado `default` .

Para crear y cambiar workspaces se usan comandos CLI.

■ Workspaces y estado

Comandos básicos de Workspaces

- Crear un nuevo workspace:

```
terraform workspace new staging
```

- Listar workspaces existentes:

```
terraform workspace list
```

- Cambiar al workspace deseado:

```
terraform workspace select staging
```

- Ver el workspace activo:

```
terraform workspace show
```

■ Comandos CLI relacionados

Ejemplo de uso de workspaces para entornos

Supón que tienes una infraestructura definida en `main.tf`. Puedes gestionar tres entornos (`dev` , `qa` , `prod`) así:

```
terraform workspace new dev
terraform apply # Aplica recursos al entorno dev
```

```
terraform workspace new prod
terraform apply # Aplica recursos al entorno prod
```

Cada entorno mantiene su propio `terraform.tfstate` y sus propios recursos, aunque compartan el mismo código.

Ejemplo multientorno

Integración de workspace en la configuración

Puedes usar el nombre del workspace dentro del código:

```
resource "aws_s3_bucket" "logs" {
  bucket = "logs-${terraform.workspace}"
  acl     = "private"
}
```

Esto permite crear recursos separados por entorno sin necesidad de múltiples archivos.

Uso de `terraform.workspace`

Consideraciones al usar workspaces

-  Útiles para entornos con configuraciones idénticas y recursos aislados.
-  No sustituyen a módulos ni estructuras separadas si hay lógica muy diferente entre entornos.
-  Algunos backends (como S3) almacenan el estado de cada workspace en una clave distinta.

Por ejemplo, S3 usa:

```
s3://bucket/key/default/terraform.tfstate
s3://bucket/key/prod/terraform.tfstate
```

Limitaciones y patrones

Módulos, Expresiones y Funciones

Módulos reutilizables

Un **módulo** en Terraform es una agrupación de recursos y lógica reutilizable. Cualquier carpeta con archivos `.tf` puede considerarse un módulo.

Ventajas:

- **Reutilización**: mismo módulo se puede usar con diferentes inputs.
- **Organización**: cada módulo encapsula lógica específica (redes, compute, seguridad).
- **Escalabilidad**: facilita la expansión y mantenimiento de grandes infraestructuras.
- **Colaboración**: equipos diferentes pueden trabajar sobre módulos distintos.

Puedes llamar a un módulo desde un proyecto principal usando el bloque `module`.

Los módulos pueden ser locales o remotos (Terraform Registry, Git, HTTP, etc.).

Conceptos de módulos

Diseño de módulos

Estructura interna de un módulo

Un módulo bien estructurado debe incluir:

- `main.tf` : lógica principal (recursos)
- `variables.tf` : definición de variables de entrada
- `outputs.tf` : valores exportados

Esto mejora la claridad y permite la reutilización en distintos entornos.

Ejemplo de estructura:

```

└─ modulo-red
   └─ main.tf
   └─ variables.tf
   └─ outputs.tf
```

Organización de módulos

Ejemplo completo de módulo en AWS

Definimos el módulo en una carpeta que lo identifica por ejemplo `s3_bucket` ...

`modulos/s3_bucket/main.tf` :

```
resource "aws_s3_bucket" "bucket" {  
  bucket = var.bucket_name  
  acl    = var.acl  
}
```

`modulos/s3_bucket/variables.tf` :

```
variable "bucket_name" { type = string }  
variable "acl"         { type = string }
```

Uso en proyecto del módulo `s3_bucket` :

```
module "logs" {  
  source      = "../modulos/s3_bucket"  
  bucket_name = "logs-dev"  
  acl        = "private"  
}
```

Ejemplo de uso local de módulo

Módulos anidados y reutilización

Terraform permite anidar módulos: un módulo puede usar otros módulos dentro.

```
module "red" {  
  source      = "../modulos/red"  
  cidr_block  = "10.0.0.0/16"  
}  
  
module "servidores" {  
  source      = "../modulos/servidores"  
  red_id      = module.red.id  
}
```

Esto facilita la composición modular de entornos completos.

Expresiones

Las **expresiones** permiten construir valores a partir de otros, como referencias, operadores o llamadas a funciones. Son fundamentales para tener una estructura ágil y que se pueda reutilizar.

Ejemplos:

```
var.region           # Referencia
"${var.nombre}-bucket" # Interpolación
var.entorno == "prod" # Expresión booleana
length(var.lista) > 2  # Composición
```

Se utilizan dentro de recursos, variables, locals y outputs.

■ Expresiones en Terraform

Tipos de expresiones

Terraform soporta varios tipos de expresiones:

- **Lógicas:** `&&` , `||` , `!`
- **Condicionales:** `condition ? true_val : false_val`
- **Interpolación de cadenas**
- **Operaciones matemáticas:** `+` , `-` , `*` , `/`
- **Expresiones de colección:** `for` , `for_each` , `count`

Estas permiten crear configuraciones dinámicas, iterar listas y definir comportamientos condicionales.

■ Tipos de expresiones

Expresiones condicionales

Permiten seleccionar valores según una condición:

```
variable "es_produccion" {
  type = bool
}

resource "aws_instance" "web" {
  instance_type = var.es_produccion ? "t3.large" : "t3.micro"
}
```

Útil para cambiar comportamiento según entorno.

■ Condicionales

Expresiones `for` y `for_each`

La expresión `for` permite transformar listas o mapas:

```
[for nombre in var.usuarios : upper(nombre)]
```

`for_each` se usa en recursos:

```
resource "aws_s3_bucket" "usuarios" {
  for_each = toset(var.nombres)

  bucket = "${each.key}-bucket"
}
```

Permiten crear múltiples recursos dinámicamente según una colección.

■ `for`

Uso avanzado de `for` en objetos

Se pueden crear mapas dinámicos con `for` :

```
locals {
  tags = {
    for nombre in var.etiquetas :
    nombre => upper(nombre)
  }
}
```

O listas con condiciones:

```
[for v in var.lista : v if v != ""]
```

Permite lógica avanzada y limpieza de datos.

■ Expresiones for

Funciones

Terraform incluye una librería extensa de **funciones** integradas:

- **Cadena:** join , replace , upper
- **Números:** min , max , abs
- **Colecciones:** length , contains , merge
- **Date/time:** timestamp
- **Codificación:** base64encode , jsonencode

Ejemplo:

```
local {  
  prefijo = upper(var.entorno)  
}
```

■ Funciones integradas

Funciones con colecciones: ejemplo práctico

```
locals {  
  servidores = ["app1", "app2", "app3"]  
  etiquetas  = [for nombre in local.servidores : "srv-${upper(nombre)}"]  
}
```

Resultado:

```
["srv-APP1", "srv-APP2", "srv-APP3"]
```

Estas transformaciones son comunes para nombrar recursos o aplicar reglas.

■ Colecciones y funciones

Uso de funciones con condiciones

```
locals {  
  zona = var.entorno == "prod" ? "eu-west-1a" : "eu-west-1b"  
}
```

Permite lógica adaptativa en base al entorno. Ideal para valores predeterminados o despliegues condicionales.

■ Funciones y condiciones

Composición de funciones y expresiones

Puedes combinar funciones con expresiones para construir lógica compleja:

```
output "prefijo_final" {  
  value = upper(replace(var.nombre, " ", "-"))  
}
```

Este ejemplo:

1. Reemplaza espacios en `nombre` por guiones
2. Convierte el resultado a mayúsculas

La composición permite mantener configuraciones limpias y potentes.

■ Ejemplos de funciones

Buenas prácticas con expresiones y funciones

- ✓ Utiliza siempre `locals` para encapsular expresiones complejas
- ✓ Es mejor usar funciones integradas antes que lógica en scripts externos
- ✓ Usa `for_each` sobre `count` cuando trabajes con mapas
- ✓ Documenta las expresiones complejas con comentarios representativos de su utilidad
- ✓ Evita anidamientos excesivos: dificultan legibilidad y comprensión del proyecto

■ Recomendaciones avanzadas

Provisioners

Los **provisioners** ejecutan scripts o comandos dentro o contra una máquina creada por Terraform.

Se usan para:

- Instalar software
- Configurar servicios
- Realizar tareas posteriores al despliegue

⚠ Deben usarse solo cuando no hay alternativa nativa en el proveedor.

■ Provisioners

Tipos de provisioners

Terraform soporta los siguientes provisioners:

- `local-exec` : ejecuta comandos localmente
- `remote-exec` : ejecuta comandos en la máquina provisionada
- `file` : copia archivos desde la máquina local a la remota

Cada uno tiene una sintaxis y uso específico según el caso.

■ Tipos de provisioners

Ejemplo de `local-exec`

```
resource "null_resource" "local" {  
  provisioner "local-exec" {  
    command = "echo ${var.mensaje} > salida.txt"  
  }  
}
```

Este comando se ejecuta **en la máquina local** donde se lanza Terraform.

Útil para generar archivos, lanzar scripts o registrar logs.

■ `local-exec`

Ejemplo de remote-exec

```
resource "aws_instance" "ejemplo" {  
  ami          = var.ami  
  instance_type = "t2.micro"  
  
  provisioner "remote-exec" {  
    inline = [  
      "sudo apt update",  
      "sudo apt install -y nginx"  
    ]  
  }  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"  
    private_key = file("~/ssh/id_rsa")  
    host      = self.public_ip  
  }  
}
```

Este script se ejecuta **dentro de la instancia AWS**.

 remote-exec

Provisioner file

```
resource "aws_instance" "ejemplo" {  
  ami          = var.ami  
  instance_type = "t2.micro"  
  
  provisioner "file" {  
    source      = "app.conf"  
    destination = "/etc/app.conf"  
  }  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"  
    private_key = file("~/ssh/id_rsa")  
    host      = self.public_ip  
  }  
}
```

Permite copiar archivos desde local hacia la máquina remota.

Provisioner file

Provisioners y dependencias implícitas

Terraform crea automáticamente dependencias entre recursos y provisioners.

Si el `remote-exec` falla, el recurso puede marcarse como fallido.

Puedes controlar la ejecución con `when = create | destroy`.

```
provisioner "remote-exec" {  
  when = destroy  
  inline = ["echo 'Destruyendo VM'"]  
}
```

Ejecutar al destruir

Buenas prácticas con provisioners

- ✓ Hay que usar `remote-exec` solo si no hay alternativa declarativa
- ✓ Es mejor utilizar `user_data` , `cloud-init` o extensiones nativas de proveedor
- ✓ Ojo, hay qye controlar errores y dependencias explícitamente
- ✗ No abuses: al final reduce portabilidad y reproducibilidad del plan

 [Recomendaciones de uso](#)